

AGENT CREW

Agent Crew 1.0

产品功能与架构设计

当前实现基线 · 面向产品负责人、技术负责人和研发测试人员

文档版本	发布日期	文档状态
1.0.1	2026-07-16	1.0 功能型 MVP 验收基线

阅读提示

本文档描述当前已经实现并通过真实全新会话、纯界面端到端验收的 Agent Crew 1.0 功能型 MVP。该结果证明核心生产闭环可用；正式 1.0 发布仍需完成环境可复现、干净基线复验和持续稳定性收口。

目录

1. 文档说明与版本信息

本文档是 Agent Crew 1.0 的产品功能和架构设计基线。它统一说明产品解决的问题、用户决策边界、完整功能、系统模块、运行逻辑、数据结构、部署依赖和验收状态。

项目	内容
产品名称	Agent Crew
文档版本	1.0.1
实现基线	2026-07-16 当前工作区代码
当前状态	功能型 MVP 已完成真实产品闭环和用户验收；进入 1.0 发布收口
主要读者	产品负责人、技术负责人、开发人员、测试人员

事实口径

“已实现”仅指当前代码和验证事实支持的能力；不把账号、团队协作、计费、多租户、云端高可用、桌面端或移动原生应用写入 1.0 现有能力。

2. 产品概述

2.1 产品定位

Agent Crew 是面向单兵产品开发者的多 Agent 协作工具。用户只与产品经理 Agent 沟通；系统先把确认后的产品方向转化为版本化 ProductContract，再生成并强制评审 InteractionContract，随后组织架构、开发、测试、审查和交付 Agent 完成一个可运行、可验证、可留档的浏览器 Web 产品。

2.2 目标用户

- 需要把产品想法落地为可运行 Web 产品的独立开发者或产品负责人。
- 希望由一个统一的 PM 入口管理需求、范围、开发过程和验收的人。
- 关注最终产品是否真正可用，同时要求开发阻塞可诊断、可干预、可追溯的人。

2.3 核心价值

价值	1.0 的实现方式
----	-----------

价值	1.0 的实现方式
先确认做什么	PM 生成版本化 ProductContract；用户批准前不进入开发。
先确认产品如何交互	InteractionDesigner 生成 InteractionContract；需求覆盖和可执行性评审通过后才进入工程规划。
让多 Agent 围绕同一目标工作	Reviewer 审查 DeliveryPackage，任务共享产品合同、交互合同、验收绑定和 Session Workspace。
以真实产品行为而非代码数量验收	Docker 启动产品，Playwright 按合同用户旅程执行自动验收。
失败时可控	产品缺陷自动修复；基础设施、非产品缺陷或修复超限形成 BlockerReport。
交付可独立复核	生成合同、计划、事件、状态流、证据、运行手册、校验和与 ZIP。

2.4 1.0 范围

包含：浏览器 Web 产品开发；产品合同确认；交互合同生成与强制评审；多 Agent 工程执行；Docker 隔离运行；自动验收与修复；阻塞人工决策；独立窗口最终用户验收；开发留档。

不包含：账号登录、团队权限、云端多租户、计费、移动原生应用、桌面客户端、任意部署平台和高可用集群。

3. 用户角色与典型场景

角色	主要目标	在系统中的行为
产品发起人	把真实产品需求变为可用产品	新建会话、描述需求、确认产品方向、最终验收
验收人	确认产品满足批准合同	执行用户旅程、验收通过或报告偏差
阻塞决策人	在系统无法自动推进时做取舍	查看证据、影响和方案，选择恢复、修订合同或取消
维护者	运行 Agent Crew 并检查留档	配置环境、启动服务、检查状态与可审计交付包

3.1 典型使用场景

- 从一个新需求开始：用户通过 PM 澄清需求，确认产品方向后让系统自动推进。
- 对产品方向提出修改：用户不批准草案，提交范围、目标用户或验收口径的修改意见。
- 处理开发阻塞：系统展示问题、证据、影响和处理方案，用户明确选择后从检查点继续。
- 验收真实产品：自动验收通过后，用户打开产品按合同旅程操作，决定接受或报告偏差。
- 留存交付依据：完成后下载 ZIP，独立检查合同、执行、证据、验收记录和校验和。

4. Agent Crew 1.0 完整功能

4.1 产品决策与合同管理

功能	用户可见行为	系统行为
需求发现	用户通过 PM 描述产品目标	PMStateMachine 维护发现与规划状态
澄清与去重	PM 只追问影响方向和验收的问题	重复问题检测与轮次约束避免无效循环
产品方向草案	展示问题、目标用户、预期结果、范围和约束	ContractBuilder 生成结构化 ProductContract
用户旅程与验收	用户看到必须通过的旅程和结果	合同保存 acceptance_scenarios 并进入后续验收绑定
批准与修改	批准后开始开发；不批准则提交修改意见	批准写入时间；修改创建下一版草案
版本化合同	每轮方向变化都有独立版本	DeliveryPackage 和最终验收均绑定合同版本

4.2 交互设计与强制评审

功能	用户可见行为	系统行为
交互合同生成	产品方向批准后，页面显示交互设计与评审进度	InteractionDesigner 将页面、控件、导航、反馈和验收步骤写入 InteractionContract
需求覆盖校验	交互范围与产品合同保持一致	Requirement Matrix 检查必须场景和关键能力是否均有交互承载
可执行性评审	不可实现或不可验收的交互不会进入开发	Reviewer 校验控件 role、可访问名称、导航和验收步骤绑定
自动复审	评审不通过时系统显示正在修订	Designer 按评审证据修订候选合同并重新提交，受最大轮次约束
人工阻塞	自动复审仍失败时显示原因、影响和处理方案	生成 BlockerReport，用户可重试、修订产品合同或取消

4.3 多 Agent 工程协作

功能	说明
交付计划	Architect 与 DeliveryPlanner 将已批准的产品合同和交互合同转成任务、依赖、Manifest 和验收绑定。
强制审查	Reviewer 未批准工程包时不进入开发；审查失败重新规划或形成阻塞。

功能	说明
DAG 调度	CrewRunner 按依赖启动任务；同一 Agent 同轮串行，不同就绪 Agent 可并行。
共享工作区	每个 session 使用 projects/<session_id>，Agent 对同一项目文件协作。
上下文传递	下游任务获得项目摘要、已批准合同、上游产物和工作区文件上下文。
工程角色	前端、后端、设计、DevOps、测试和审查角色按计划承担不同任务。
任务过程可见	前端通过 SSE 显示状态变化、工具调用、任务完成和错误事件。

4.4 质量验证与真实交付

阶段	检查内容	失败处理
预检	Manifest 运行时、命令、端口、产物和环境要求	不可执行则阻塞
准备	在 Docker 隔离环境执行安装或构建命令	产品缺陷进入修复；基础设施问题阻塞
启动	按 Manifest 启动容器并映射临时端口	启动失败形成基础设施 BlockerReport
健康检查	访问约定路径并检查状态码	根据失败类别修复或阻塞
浏览器验收	Playwright 按 AcceptanceBinding 执行 open/click/fill/expect	保存失败证据并归因
自动修复	将产品缺陷派回任务 owner，修复后重新验收，最多 3 次	次数耗尽形成阻塞
待用户验收	所有 must 场景通过后在独立窗口开放真实产品	系统不能代替用户进入 completed

4.5 人工干预与恢复

系统不能自动解决的问题必须显式进入 blocked，而不是长时间静默等待。BlockerReport 包含问题类别、摘要、诊断证据、影响、可选方案、推荐方案和恢复检查点。用户选择后，系统从交互设计、工程规划、开发、验证或修复的合法检查点恢复，或者创建下一版产品合同。

4.6 会话、状态与事件

- 会话创建、命名、切换和历史消息读取。
- Agent 在线状态、工作状态、错误状态和全局取消。
- SSE 实时事件流；产品状态和留档状态定时刷新。
- 长对话可执行上下文压缩，降低模型输入体积。

4.7 可审计交付

Agent Crew 在产品工作区生成独立的 agentcrew-delivery 目录和同名 ZIP。留档不是数据库状态的截图，而是可携带、可解析、可复核的交付投影。

留档文件	用途
product-contract.json	用户批准的产品方向、范围、旅程和验收场景
interaction-contract.json	页面、控件、导航、反馈、需求覆盖和可执行验收步骤
delivery-plan.json	Reviewer 批准后的工程任务、Manifest 和验收绑定
agent-execution.json	当前 session 的真实任务事件
state-flow.json	状态版本和完整 flow log
blocker-decisions.json	阻塞报告、用户选择和恢复记录
automatic-acceptance.json	逐场景状态和成功/失败证据
user-acceptance.json	pending、accepted 或 rejected 及合同版本
runbook.md	运行时、命令、健康检查和验收说明
checksums.sha256	最终产品 artifacts 的 SHA-256

5. 功能架构设计

功能架构按用户入口、核心功能、平台服务、数据资产和运行环境分层。核心功能围绕“先确认产品方向、再强制评审交互、组织工程生产、真实验收、失败时人工决策”的闭环划分。下一页给出完整功能架构图。



图 5-1 Agent Crew 1.0 功能架构（当前实现基线）

5.1 功能域说明

功能域	边界	主要输出
产品决策中心	只决定做什么和如何验收，不直接执行工程任务	版本化 ProductContract
交互与工程中心	先把产品合同转成交互合同并强制评审，再生成工程任务并执行	InteractionContract、DeliveryPackage、源码、测试和任务结果
质量与交付中心	隔离运行产品并按用户旅程验证	Preview、自动验收结果和修复记录
人工干预与治理中心	处理系统不能安全自动决定的问题	BlockerReport、用户决策和恢复记录
可审计交付	将状态、证据和产品 artifacts 投影为独立档案	agentcrew-delivery 目录与 ZIP

6. 逻辑架构设计

逻辑架构以领域职责而不是代码目录分层：交互接口负责接收请求和展示状态；产品决策域形成 ProductContract；交互设计与强制评审域形成 InteractionContract；规划与编排域建立可执行方案；Agent 执行域生产文件；隔离交付域启动并验证产品；持久化与审计域保存事实并生成留档。下一页展示当前真实组件及关系。

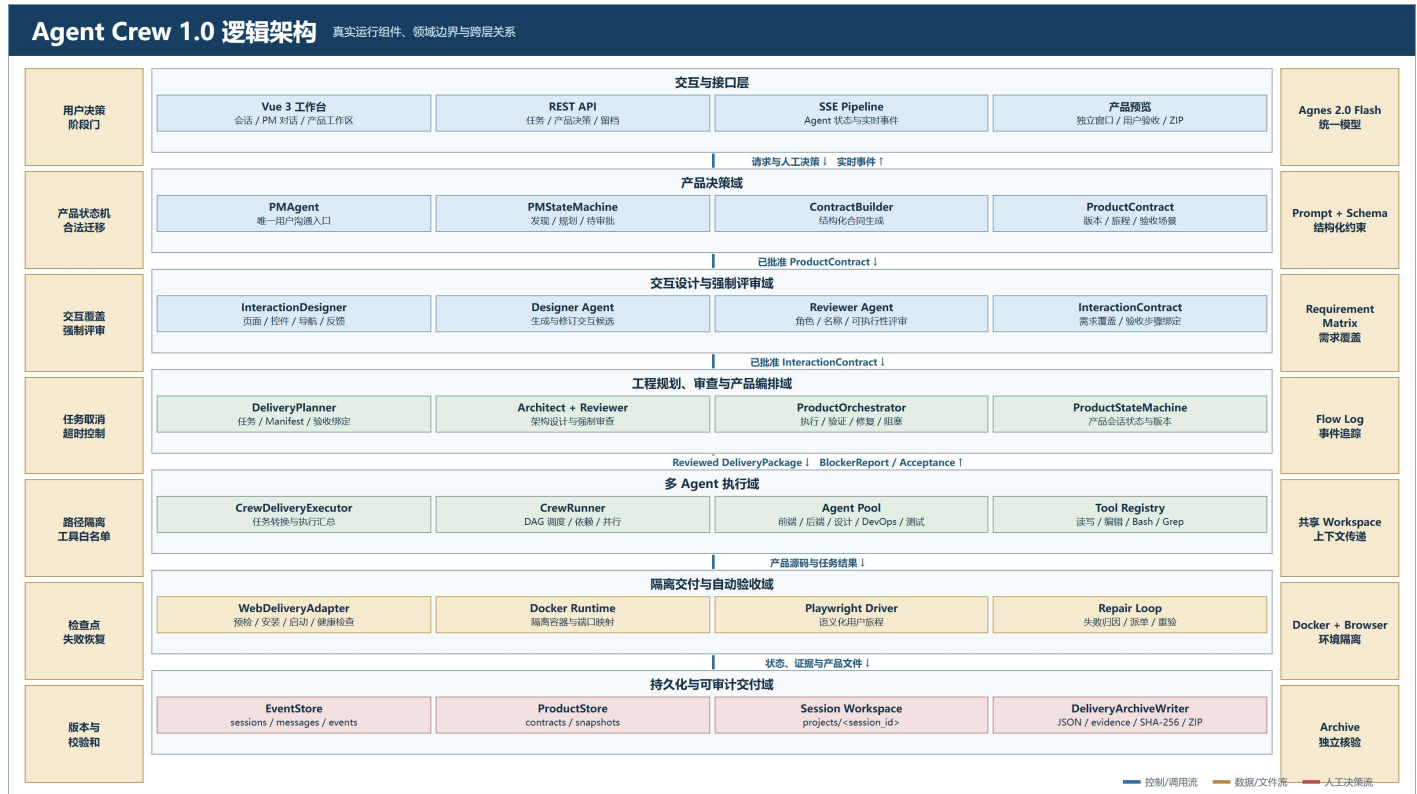


图 6-1 Agent Crew 1.0 逻辑架构（当前实现基线）

6.1 关键组件职责

组件	职责	关键依赖
PMAgent / PMStateMachine	管理用户沟通、需求发现、合同确认和旧计划审批入口	LLMProxy、ProductStateMachine
ContractBuilder / ProductContract	生成并校验版本化产品合同	结构化模型输出、ProductStore
InteractionDesigner / InteractionContract	生成页面、控件、导航与反馈合同，并完成需求覆盖和强制评审	Designer Agent、Reviewer Agent、Requirement Matrix
DeliveryPlanner	基于产品合同与交互合同生成 DeliveryPackage，并要求 Architect/Reviewer 审查	Agent Pool、DeliveryManifest
ProductOrchestrator	执行状态流、修复循环、阻塞恢复和最终验收	CrewDeliveryExecutor、WebDeliveryAdapter、ProductStore
CrewRunner	管理 Agent 生命周期、DAG 依赖、并发和工作区上下文	Agent Pool、Tool Registry、EventStore
WebDeliveryAdapter	Docker 预检、准备、启动、健康检查和 Playwright 验收	Docker Desktop、Chromium
DeliveryArchiveWriter	将合同、计划、事件、状态、证据和校验和写入档案及 ZIP	EventStore、Session Workspace

6.2 三类关键流

控制/调用流：用户请求从 Vue/REST 进入 PM 和产品编排层，再向 CrewRunner、交付适配器和持久化组件下发。

数据/文件流：ProductContract 先形成 InteractionContract，再变为 DeliveryPackage；Agent 在 Session Workspace 生产文件；验证证据和状态最终进入交付档案。

人工决策流：合同批准、阻塞方案和最终验收从系统返回用户，用户决定后再进入下一合法状态。

7. 端到端功能流程

端到端流程包含三个不可由模型越过的用户阶段门：产品合同批准、阻塞方案选择和最终用户验收。产品合同批准后，系统依次完成交互合同强制评审、工程包强制评审、任务调度、容器运行、浏览器验收和自动修复。下一页给出六阶段正常路径和异常分支。

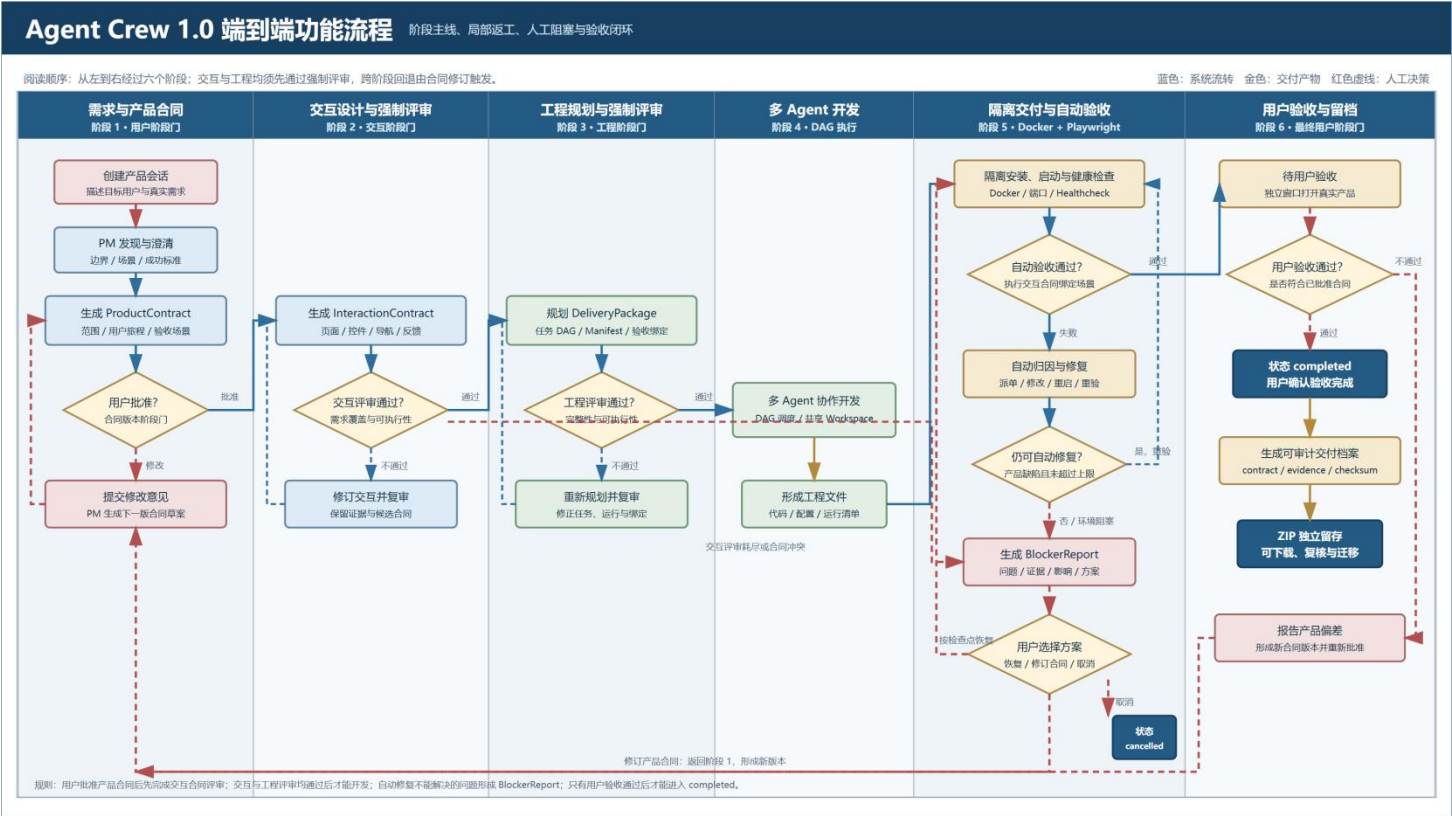


图 7-1 Agent Crew 1.0 端到端功能流程（当前实现基线）

7.1 流程关键规则

- 产品合同未批准时，系统不得创建正式 DeliveryPackage 或开始工程执行。
- InteractionContract 未通过需求覆盖与可执行性强制评审时，系统不得进入工程规划。
- Reviewer 未批准工程包时，系统不得进入多 Agent 开发。
- 只有产品缺陷且修复次数未超限时才自动修复；基础设施故障和不可归因问题进入 blocked。
- 自动验收成功只进入 ready_for_acceptance；只有用户明确确认后才进入 completed。
- 用户报告偏差或选择修改合同时，形成下一版产品合同，而不是继续修改已经批准的旧版本。
- 状态、证据和用户决策随执行快照刷新到交付档案。

8. 状态机与人工决策边界

8.1 产品会话状态

状态	含义	主要下一状态
discovering	PM 正在发现或修订产品方向	awaiting_contract_approval
awaiting_contract_approval	等待用户批准合同草案	planning 或 discovering
designing_interaction	生成或修订交互合同	reviewing_interaction / blocked
reviewing_interaction	强制评审需求覆盖和交互可执行性	designing_interaction / planning / blocked
planning	构建交付计划；首次进入时可先转交互设计	designing_interaction / reviewing_plan / running / blocked
reviewing_plan	审查工程包	running / blocked
running	多 Agent 执行工程任务	verifying / blocked
verifying	准备、启动并执行自动验收	fixing / ready_for_acceptance / blocked
fixing	修复已归因的产品缺陷	verifying / blocked
blocked	等待用户选择处理方案	designing_interaction / planning / running / verifying / fixing / cancelled

状态	含义	主要下一状态
ready_for_acceptance	自动验收通过，等待用户验收	completed / blocked
completed	用户已接受产品	终态
cancelled	用户或系统取消	终态

8.2 产品执行状态

产品会话状态机覆盖 discovering、awaiting_contract_approval、designing_interaction、reviewing_interaction、planning、reviewing_plan、running、verifying、fixing、blocked、ready_for_acceptance、completed 和 cancelled。ProductOrchestrator 负责工程执行子状态并维护 allowed transitions；每次持久化增加 state_version，防止旧版本覆盖新状态。

8.3 人工决策点

决策点	用户输入	系统结果
产品合同	批准或修改意见	批准进入 planning；修改创建下一版草案
执行阻塞	选择推荐修复、重试验证、修订合同或取消	记录 blocker decision 并从检查点恢复或重启产品方向
最终验收	验收通过或报告偏差	通过进入 completed；偏差形成 rejected 记录和新合同版本

9. 核心数据设计

数据对象	核心内容	持久化位置
ProductContract	合同 ID、版本、状态、产品简报、用户旅程、验收场景、批准时间	product_contracts
InteractionContract	页面、控件、导航、反馈、需求覆盖矩阵、验收步骤绑定和评审状态	execution snapshot / interaction-contract.json
DeliveryPackage	摘要、任务、DeliveryManifest、AcceptanceBinding、ReviewDecision	execution snapshot 与交付档案
BlockerReport	问题类别、证据、影响、选项、推荐项、恢复检查点	execution snapshot / blocker-decisions.json

数据对象	核心内容	持久化位置
Execution Snapshot	状态、state_version、合同、交付包、修复、flow log、用户验收	execution_snapshots
Event	Agent、task、事件类型、摘要、结构化 detail 和时间	events / agent-execution.json
Session Workspace	产品源码、测试、文档、证据和交付档案	projects/<session_id>

9.1 DeliveryManifest

- 声明 delivery_type、runtime、container_port、安装命令、启动命令和健康检查。
- 声明 verification_commands、required_environment 和最终 artifacts。
- 所有路径必须是安全的相对路径；命令采用 argv 结构，避免依赖未解析的 shell 字符串。

9.2 AcceptanceBinding

- DeliveryPackage 的验收绑定优先继承已批准 InteractionContract 中的可执行场景。
- 每个 must 验收场景必须有绑定，且 contract_id 和 contract_version 必须匹配。
- 浏览器步骤支持 open、click、fill 和 expect，并使用 role、label、text 等语义定位。
- 成功场景保存 <scenario_id>-passed.png；失败保存 <scenario_id>-failed.png。

9.3 可审计交付目录

项目内交付合同

projects/<session_id>/agentcrew-delivery/ 包含 10 个必需文件和 evidence/；目录旁生成 agentcrew-delivery.zip。checksums.sha256 只覆盖 DeliveryManifest 声明的最终产品 artifacts。

10. Agent 角色与职责

角色	主要职责	典型能力/工具	主要产出
PM	用户沟通、需求发现、产品方向与阶段门汇报	模型调用、会话状态、任务分派	ProductContract、决策提示
Architect	把合同转为技术架构与工程约束	无直接写代码工具；输出结构化设计	架构方案、Manifest 候选
Reviewer	强制审查交互合同、工程包和最终实现风险	独立结构化审查	交互评审结论、ReviewDecision、审查发现

角色	主要职责	典型能力/工具	主要产出
Frontend	实现浏览器交互、样式和前端状态	read_file、edit_file	HTML/CSS/JavaScript
Backend	实现后端 API、数据和业务逻辑	read_file、edit_file、bash、grep	Python 服务和测试
Designer	定义页面、控件、导航、反馈和视觉结构	模型调用、read_file、write_file	InteractionContract、设计说明和视觉文件
DevOps	容器、运行环境和交付配置	读写编辑、bash、grep	容器与运行配置
Tester	执行验证、定位缺陷并提供证据	read_file、bash、grep	测试结果和失败证据

10.1 协作约束

- 所有工程角色围绕同一 ProductContract、InteractionContract 版本和 DeliveryPackage 工作。
- 任务 dependencies 决定启动顺序；工作区文件和项目摘要构成跨 Agent 上下文。
- 验证失败按 owner 派回可修复角色；不存在合适 owner 时形成阻塞而不是任意改派。
- 用户不需要逐个指挥工程 Agent；用户只在产品方向、阻塞和验收阶段做决定。

11. 接口与事件设计

11.1 REST API 分组

接口组	代表接口	用途
任务与消息	POST /tasks; GET /tasks/{id}; GET /messages/{agent_id}	发起 PM/Agent 任务并读取结果
会话	POST /sessions; GET /sessions	创建、列出和切换会话
产品状态	GET /api/product/status	读取合同、交付包、阻塞和预览状态
合同决策	POST /api/product/contract/approve; /revise	批准或修订产品合同
阻塞决策	POST /api/product/blocker/decide	选择处理方案并恢复
最终验收	POST /api/product/accept; /reject	接受产品或报告偏差
开发留档	GET /api/product/archive/status; /download	查看完整性并下载 ZIP
运行控制	POST /api/agent/cancel; /api/cancel-all	取消单个或全部 Agent

接口组	代表接口	用途
统计与健康	GET /llm/stats; GET /health	读取模型用量与服务状态

11.2 SSE 事件

前端通过 /events/stream 建立 EventSource。事件包含 event_id、agent_id、task_id、event_type、summary、detail 和 timestamp。主要类型包括 task_start、task_complete、tool_call、status_change 和 error。

11.3 前端同步策略

- SSE 用于实时任务和 Agent 事件。
- 产品状态通过 /api/product/status 轮询，避免只依赖内存事件。
- 进入 ready_for_acceptance 或 completed 后同步读取留档完整性。

12. 运行与部署架构

层	当前实现
前端	Vue 3 CDN、原生 JavaScript 和 CSS，由 FastAPI 提供静态资源
后端	Python 3.11+、FastAPI、asyncio；主要入口 backend/app.py
数据库	本地 SQLite crew_events.db，保存会话、消息、事件、合同和执行快照
项目存储	本地文件系统 projects/<session_id>，每个会话独立工作区
模型服务	当前统一使用 Agnes 2.0 Flash 的 OpenAI-compatible 接口；地址、模型和凭据由环境变量配置，凭据不进入源码或留档
隔离运行	Docker Desktop；生成产品在容器内安装、启动和接受健康检查
浏览器验收	Playwright + Chromium，执行合同绑定的语义化用户旅程

12.1 运行拓扑

用户浏览器访问 Agent Crew Web 工作台；FastAPI 提供 REST、SSE 和静态页面；PMAgent 与 ProductOrchestrator 调用模型服务和 Agent Pool；CrewRunner 操作 session 工作区；WebDeliveryAdapter 调用 Docker 与 Playwright；EventStore 和 ProductStore 将状态写入同一 SQLite；DeliveryArchiveWriter 将可交付事实投影到项目目录。

12.2 当前本地环境

- Agent Crew 服务默认在本机端口运行；正式验收入口为 `http://127.0.0.1:8010`。
- Docker Desktop 应用位于 `J:\docker\app`，数据目录位于 `J:\docker\data`。
- 产品预览端口由运行时动态映射，并在独立浏览器窗口打开；不嵌入 Agent Crew 主工作台，也不应写死固定交付地址。

13. 安全、可靠性与可追溯性

控制项	实现方式	目的
路径隔离	Session Workspace + 相对路径校验 + resolve/relative 检查	防止 Agent 或留档越界访问
工具白名单	每个 Agent 只获得职责所需工具	降低无关写入和命令风险
命令控制	危险命令过滤、argv 命令合同和 Docker 隔离	降低宿主机破坏风险
循环与超时	工具轮次、重复调用检测、模型超时和任务超时	避免无休止执行
状态合法性	ProductStateMachine 与 ProductOrchestrator allowed transitions	阻止跳过阶段门
乐观版本	execution snapshot state_version	避免旧状态覆盖新状态
失败恢复	resume_checkpoint、blocker history 和持久化 flow log	支持人工决定后继续
证据与校验	成功/失败截图、验收记录和 SHA-256	证明行为并检测交付后篡改
秘密保护	API 凭据只来自环境变量；档案不导出完整 prompt 或密钥	避免敏感信息泄漏

14. 1.0 验收标准与当前状态

14.1 Agent Crew 1.0 通过标准

- 从全新会话开始，用户只操作 Agent Crew 界面。
- 用户能够确认或修改产品方向，未批准前不启动开发。
- 产品合同批准后生成 InteractionContract，需求覆盖和可执行性强制评审通过后才进入工程规划。
- 多 Agent 自动完成规划、审查、开发、启动和浏览器验收。
- 出现阻塞时，页面明确显示问题、证据、影响、方案和推荐项。

- 用户能够打开真实产品，按合同旅程操作，并决定通过或报告偏差。
- 完成后开发留档显示完整，ZIP 可下载，校验和能够验证产品 artifacts。
- 重启 Agent Crew 后，session 状态和留档入口仍可从持久化事实恢复。

14.2 当前工程验证结果

验证项	结果
自动化测试	220 passed (154 项主测试组 + 66 项模块回归)
前端与后端	frontend/app.js 语法检查和 backend compileall 通过
浏览器布局	桌面与 390px 移动视口无横向溢出
正式真实会话	ses_01d071d7: 自由职业者项目报价与利润测算器完成开发、自动验收和用户验收
验收结果	3 个 must 场景全部 passed，用户验收记录为 accepted，最终状态 completed
交付留档	10 个主文件及 evidence 共 16 个文件，artifact 校验和一致，ZIP 可下载并可复核

当前结论

Agent Crew 已通过一次由用户亲自执行的全新会话、纯界面端到端验收，核心产品生产闭环达到功能型 MVP 标准。正式 1.0 发布仍需补齐可重复安装与环境检查、干净代码基线复验、连续多会话稳定性和发布标签。

14.3 已知限制

- 当前标准交付对象是浏览器 Web 产品。
- 当前运行形态是本地单机，不提供多租户、高可用或远程托管控制面。
- 模型输出质量仍影响首轮计划和代码质量，系统依靠审查、确定性检查、真实验收和人工决策收敛。
- 产品预览依赖本机 Docker Desktop 和 Chromium 运行环境。
- 当前前端使用 Vue 3 CDN；完全离线环境需要本地化前端依赖后才能稳定运行。
- 账号、权限、团队空间、计费和商业运营能力不属于 1.0。

15. 附录

15.1 核心术语

术语	定义
ProductContract	用户批准的版本化产品方向合同，是开发和验收的共同事实源。
InteractionContract	基于产品合同生成并通过强制评审的交互合同，描述页面、控件、导航、反馈和可执行验收步骤。
DeliveryPackage	Reviewer 批准后的工程交付包，包含任务、Manifest 和 AcceptanceBinding。
DeliveryManifest	产品的运行时、命令、端口、健康检查、环境和 artifacts 合同。
AcceptanceBinding	把合同验收场景绑定为可执行浏览器步骤。
BlockerReport	系统不能安全自动推进时向用户展示的结构化阻塞报告。
Session Workspace	一个会话独占的项目文件目录。
Delivery Archive	可独立复核的合同、执行、状态、证据、验收和校验和集合。

15.2 核心代码映射

能力	主要文件
应用入口与 API	backend/app.py
PM 与产品会话状态	backend/agents/pm_agent.py; backend/crew_runner/product_state_machine.py
产品合同	backend/crew_runner/product_contract.py; contract_builder.py
交互合同与强制评审	backend/crew_runner/interaction_contract.py; interaction_designer.py; interaction_requirements.py
交付计划与审查	backend/crew_runner/delivery_planner.py; delivery_models.py
产品执行状态机	backend/crew_runner/product_orchestrator.py
Agent 调度与执行	backend/crew_runner/runner.py; crew_delivery_executor.py
隔离交付与验收	backend/crew_runner/web_delivery_adapter.py
持久化	backend/events/store.py; backend/events/product_store.py
可审计交付	backend/crew_runner/delivery_archive.py
Web 工作台	frontend/index.html; frontend/app.js; frontend/styles.css

文档结束